

Input/Output Devices

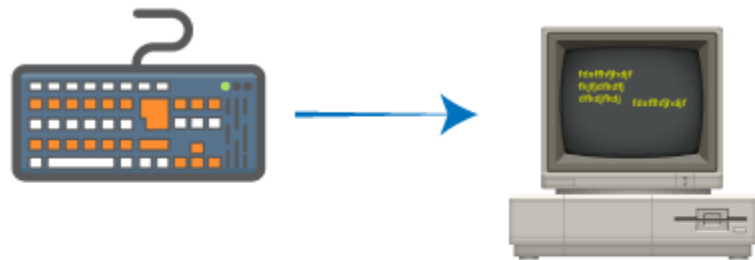
Table Contents

- Definition
- Introduction
- Input Devices
- Output Devices
- Conclusion

Definition

An input/output device, often known as an IO device, is any hardware that allows a human operator or other systems to interface with a computer

Input Example



Output Example



Introduction

- Input/output devices, as the name implies, are capable of delivering data (output) to and receiving data from a computer (input).
- An input/output (I/O) device is a piece of hardware that can take, output, or process data. It receives data as input and provides it to a computer, as well as sends computer data to storage media as a storage output.

input

output



INPUT DEVICES

Keyboard

- The keyboard is the most frequent and widely used input device for entering data into a computer. Although there are some additional keys for performing other operations, the keyboard layout is similar to that of a typical typewriter.
- Generally, keyboards come in two sizes: 84 keys or 101/102 keys, but currently keyboards with 104 keys or 108 keys are also available for Windows and the Internet.

INPUT DEVICES

Numeric Keys: It is used to enter numeric data or move the cursor.

Typing Keys: The letter keys (A-Z) and number keys (0-9) are among these keys.

Control Keys: These keys control the pointer and the screen.

Special Keys: Enter, Shift, Caps Lock, NumLk, Tab, etc., and Print Screen

Function Keys: The 12 keys from F1 to F12 on the topmost row of the keyboard.

INPUT DEVICES

Mouse

- The most common pointing device is the mouse. The mouse is used to move a little cursor across the screen while clicking and dragging.
- A mouse is an input device that lets you move the mouse on a flat surface to control the coordinates and movement of the on-screen cursor/pointer.
- The left mouse button can be used to select or move items, while the right mouse button when clicked displays extra menus.

INPUT DEVICES

Joystick

- A joystick is a pointing device that is used to move the cursor on a computer screen.
- A spherical ball is attached to both the bottom and top ends of the stick. In a socket, the lower spherical ball slides. You can move the joystick in all four directions.
- The joystick's function is comparable to that of a mouse. It is primarily used in CAD (Computer-Aided Design) and playing video games on the computer.

INPUT DEVICES

Light Pen

- A light pen is a type of pointing device that looks like a pen. It can be used to select a menu item or to draw on the monitor screen. A photocell and an optical system are enclosed in a tiny tube.
- When the tip of a light pen is moved across a monitor screen while the pen button is pushed, the photocell sensor element identifies the screen location and provides a signal to the CPU.

INPUT DEVICES

Scanner

- A scanner is an input device that functions similarly to a photocopier. It's employed when there's information on paper that needs to be transferred to the computer's hard disc for subsequent manipulation.
- Scanner collects images from the source and converts them to a digital format that may be saved on a disc. Before they are printed, these images can be modified.

INPUT DEVICES

OCR

- OCR stands for optical character recognition, and it is a device that reads printed text.
- OCR optically scans the text, character by character, turns it into a machine-readable code, and saves it to the system memory.

INPUT DEVICES

Bar Code Reader

- A bar code reader is a device that reads data that is bar-coded (data that is represented by light and dark lines).
- Bar-coded data is commonly used to mark things, number books, and so on. It could be a handheld scanner or part of a stationary scanner.

OUTPUT DEVICES

Monitor

- Monitors, also known as Visual Display Units (VDUs), are a computer's primary output device.
- It creates images by arranging small dots, known as pixels, in a rectangular pattern. The amount of pixels determines the image's sharpness.

OUTPUT DEVICES

The two kinds of viewing screen used for monitors are:

(1) Cathode-Ray Tube (CRT): Pixels are minuscule visual elements that make up a CRT display. The higher the image quality or resolution, the smaller the pixels.

(2) Flat-Panel Display Cathode-Ray Tube Monitor: In comparison to the CRT, a flat-panel display is a type of video display with less volume, weight, and power consumption.

OUTPUT DEVICES

Printer

- Printers are output devices that allow you to print information on paper. There are two types of printers:

Impact Printer:

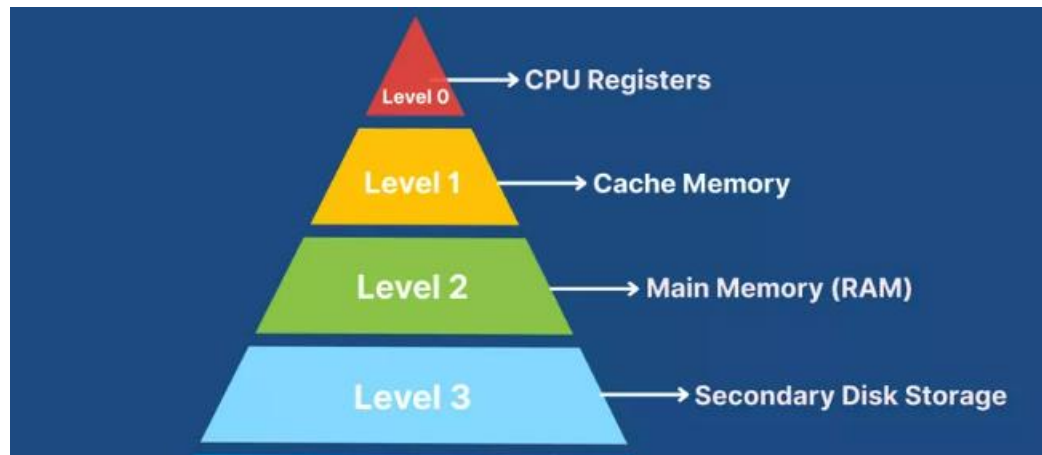
- Characters are printed on the ribbon, which is subsequently crushed against the paper, in impact printers.

Non-Impact Printers:

- Characters are printed without the need for a ribbon in non-impact printers.

Memory Hierarchy

- Memory hierarchy represents the structure of computer memory organized based on speed, cost, and size.
- Faster memories are small and expensive, placed close to CPU.
- Slower memories are larger and cheaper, placed farther away.
- Goal: balance performance and cost.



Memory Hierarchy

Level 0: CPU Registers

- **Location:** Inside CPU.
- **Speed:** Fastest memory (nanoseconds).
- **Size:** Very small (32–512 registers).
- **Function:**
 - Holds operands for immediate execution.
 - Stores results of operations.
 - Examples: Instruction Register (IR), Program Counter (PC), Accumulator.

Memory Hierarchy

Level 1: Cache Memory

- **Location:** Between CPU and Main Memory (often inside CPU chip).
- **Speed:** Faster than RAM, slower than registers.
- **Size:** Small (KBs to few MBs).
- **Function:**
 - Stores frequently accessed data/instructions.
 - Reduces time to access data from RAM.

Memory Hierarchy

Level 2: Main Memory (RAM)

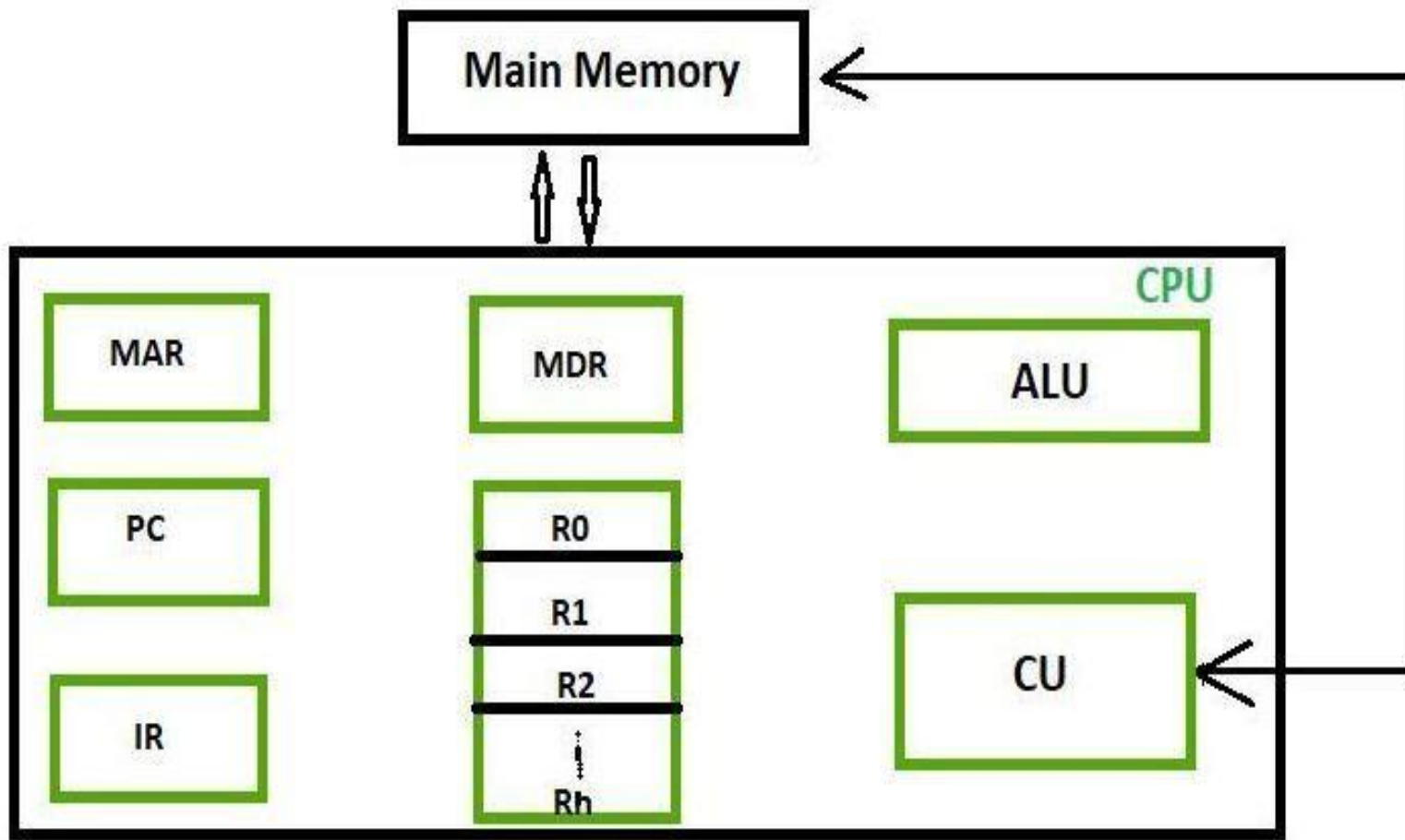
- **Location:** Outside CPU, connected via system bus.
- **Speed:** Slower than cache, faster than disk.
- **Size:** GBs (commonly 4–64 GB in modern PCs).
- **Function:**
 - Stores active programs and data.
 - Volatile → data lost when power is off.
- **Types:**
 - **DRAM (Dynamic RAM)** → slower, cheaper, used as main memory.
 - **SRAM (Static RAM)** → faster, costlier, used for cache.

Memory Hierarchy

Level 3: Secondary Storage

- **Examples:** Hard Disk (HDD), Solid State Drive (SSD).
- **Speed:** Slowest in hierarchy.
- **Size:** Huge (TBs).
- **Function:**
 - Stores OS, applications, files permanently.
 - Non-volatile → data retained even after power off.
- **SSD vs HDD:**
 - SSDs are much faster but more expensive.
 - HDDs are slower but cheaper per GB.

CPU Architecture



CPU Architecture

- **MAR (Memory Address Register):** Holds the address of memory location to be accessed.
- **MDR (Memory Data Register):** Holds data being transferred to/from memory.
- **PC (Program Counter):** Holds the address of the **next instruction** to execute.
- **IR (Instruction Register):** Holds the **current instruction** being executed.
- **AC(Accumulator):** temporarily holds intermediate results of arithmetic and logic operations performed by the ALU.
- **General Purpose Registers (R0, R1, R2...):** Temporarily hold operands, results.

CPU Architecture

ALU (Arithmetic Logic Unit)

- Performs **arithmetic operations** (add, subtract, multiply, divide).
- Performs **logical operations** (AND, OR, NOT, XOR, comparisons).
- Works on data fetched into registers.

CU (Control Unit)

- Directs the flow of data and instructions.
- Generates control signals for memory, ALU, and I/O.
- Coordinates between PC, IR, MAR, MDR, and registers.

CPU Architecture

Buses

Address Bus

Carries the **address** of memory location (from CPU → Main Memory).

Unidirectional.

Data Bus

Carries actual **data** between CPU and memory/IO devices.

Bidirectional.

Control Bus

Carries **control signals** (Read, Write, Clock, Interrupt, etc.) between CPU and other units.

Bidirectional.

CPU Architecture

Instruction Execution Cycle

1.Fetch

1. PC $\rightarrow$ MAR (address of next instruction).
2. Instruction fetched from memory into MDR $\rightarrow$ IR.
3. PC incremented.

2.Decode

1. CU decodes instruction in IR.
2. Determines required operation and operands.

3.Execute

1. ALU performs operation (using registers/operands).
2. Result stored in register or memory.

4.Store (if needed)

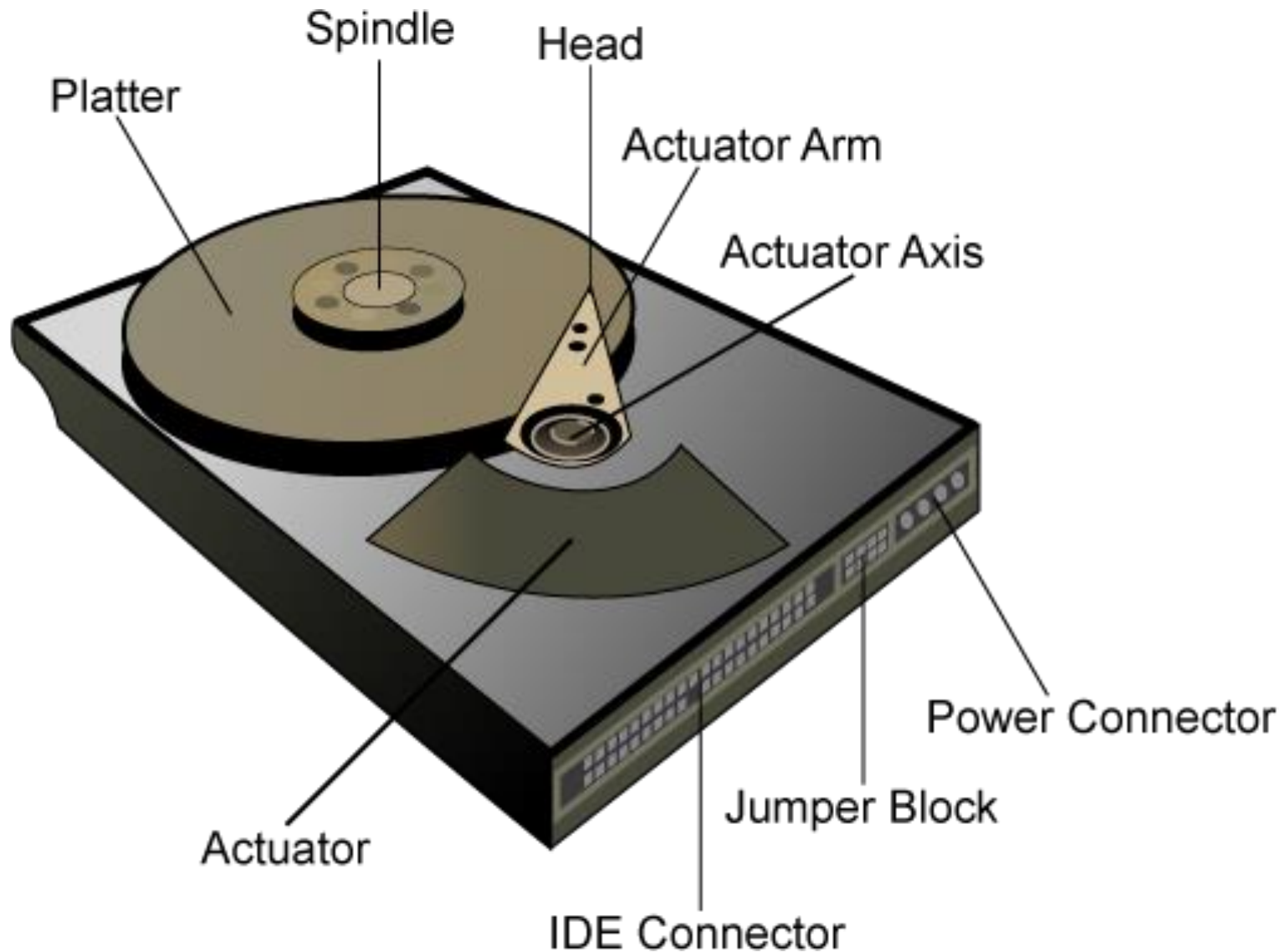
1. Result written back to memory or output device.

Storage Devices

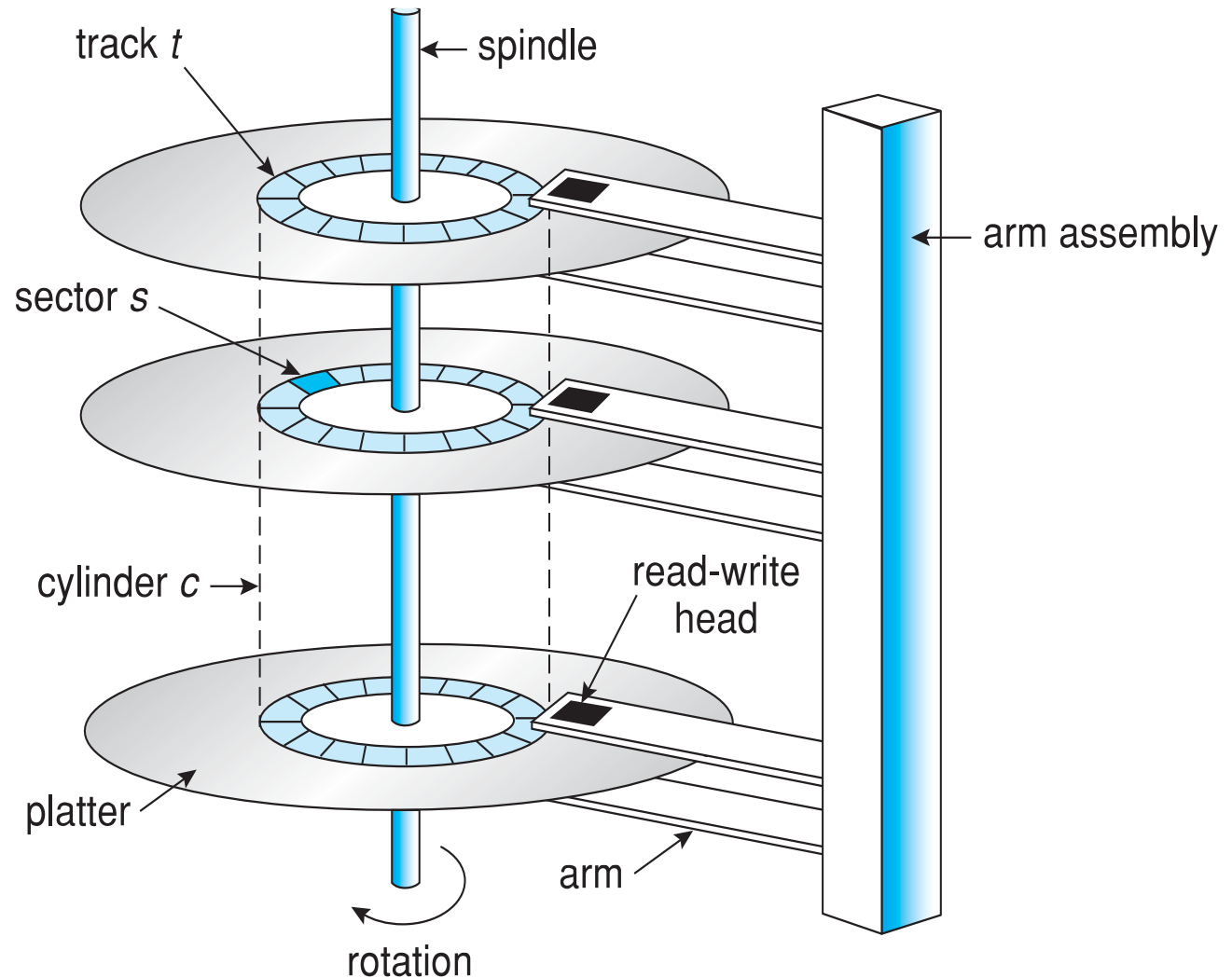
Hard Drives

SSD

Hard Drive Hardware



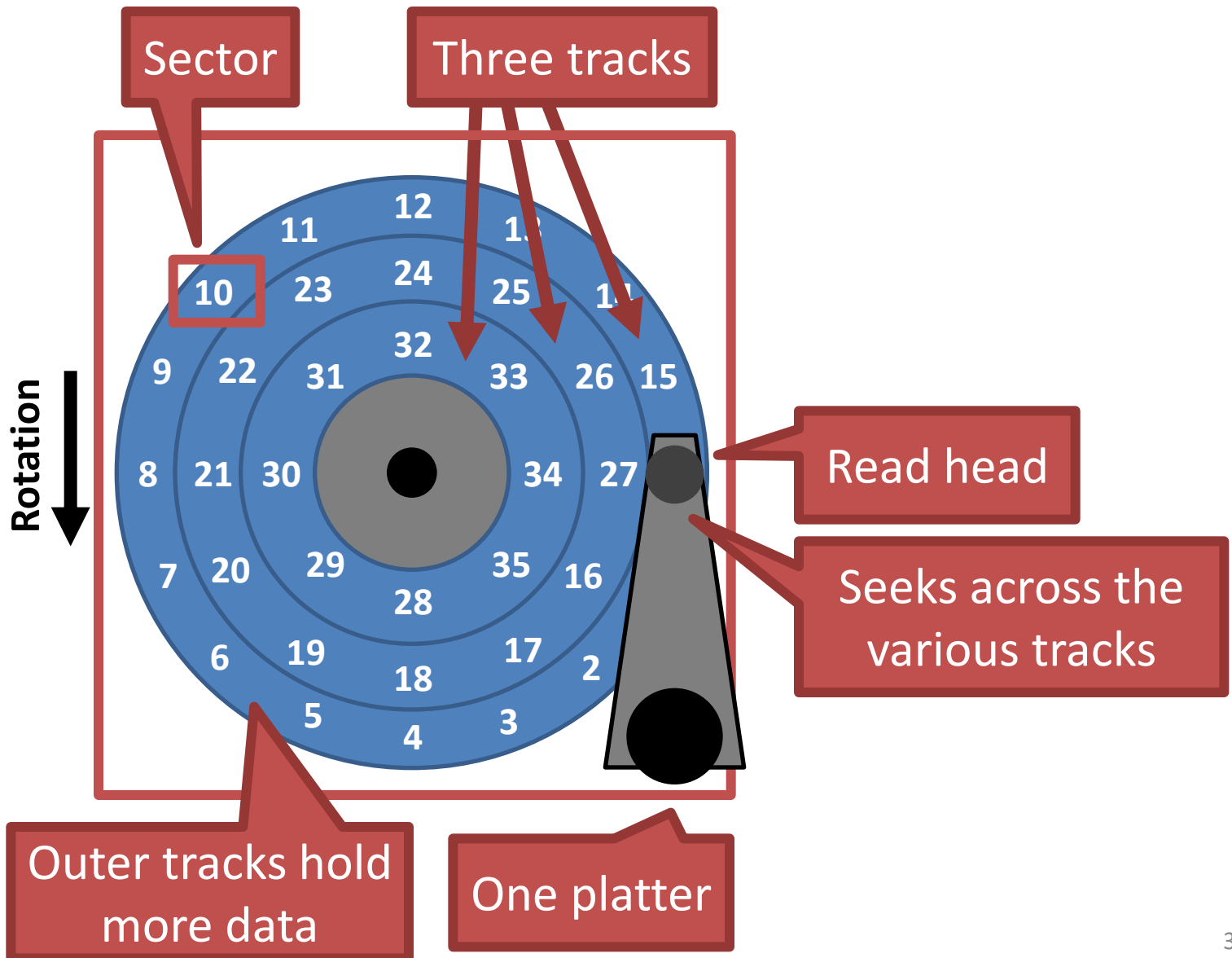
A Multi-Platter Disk



Addressing and Geometry

- Externally, hard drives expose a large number of **sectors** (blocks)
 - Typically, 512 or 4096 bytes
 - Individual sector writes are **atomic**
- Drive geometry
 - Sectors arranged into **tracks**
 - A **cylinder** is a particular track on multiple platters
 - Tracks arranged in concentric circles on **platters**
 - A disk may have multiple, double-sided platters
- Drive motor spins the platters at a constant rate
 - Measured in revolutions per minute (RPM)

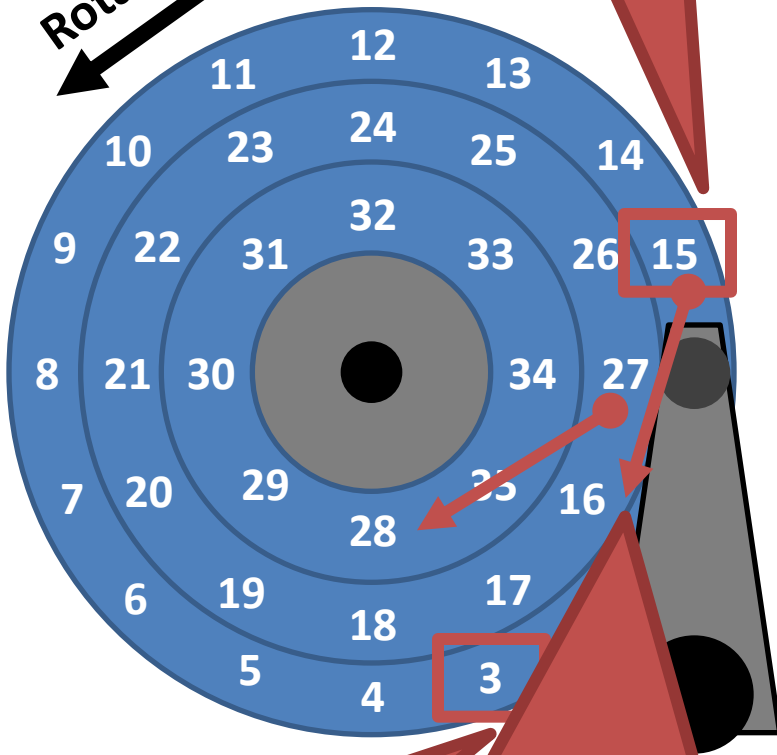
Geometry Example



Types of Delay With Disks

Long delay

Rotation



Track skew: offset sectors so that sequential reads across tracks incorporate seek delay

Three types of delay

1. Rotational Delay
 - Time to rotate the desired sector to the read head
 - Related to RPM
2. Seek delay
 - Time to move the read head to a different track
3. Transfer time
 - Time to read or write bytes

How To Calculate Transfer Time



	Cheetah 15K.5	Barracuda
Capacity	300 GB	1 TB
RPM	15000	7200
Avg. Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s

Transfer time

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

Assume we are transferring 4096 bytes

Cheetah

$$T_{I/O} = 4 \text{ ms} + 1 / (15000 \text{ RPM} / 60 \times 1000) / 2 \text{ (ms)} \\ + (4096 \text{ B} / 125 \text{ MB}) * 1000 \text{ ms}$$

$$T_{I/O} = 4 \text{ ms} + 2 \text{ ms} + 0.03125 \text{ ms} \approx 6 \text{ ms}$$

Barracuda

$$13.2 \text{ ms}$$

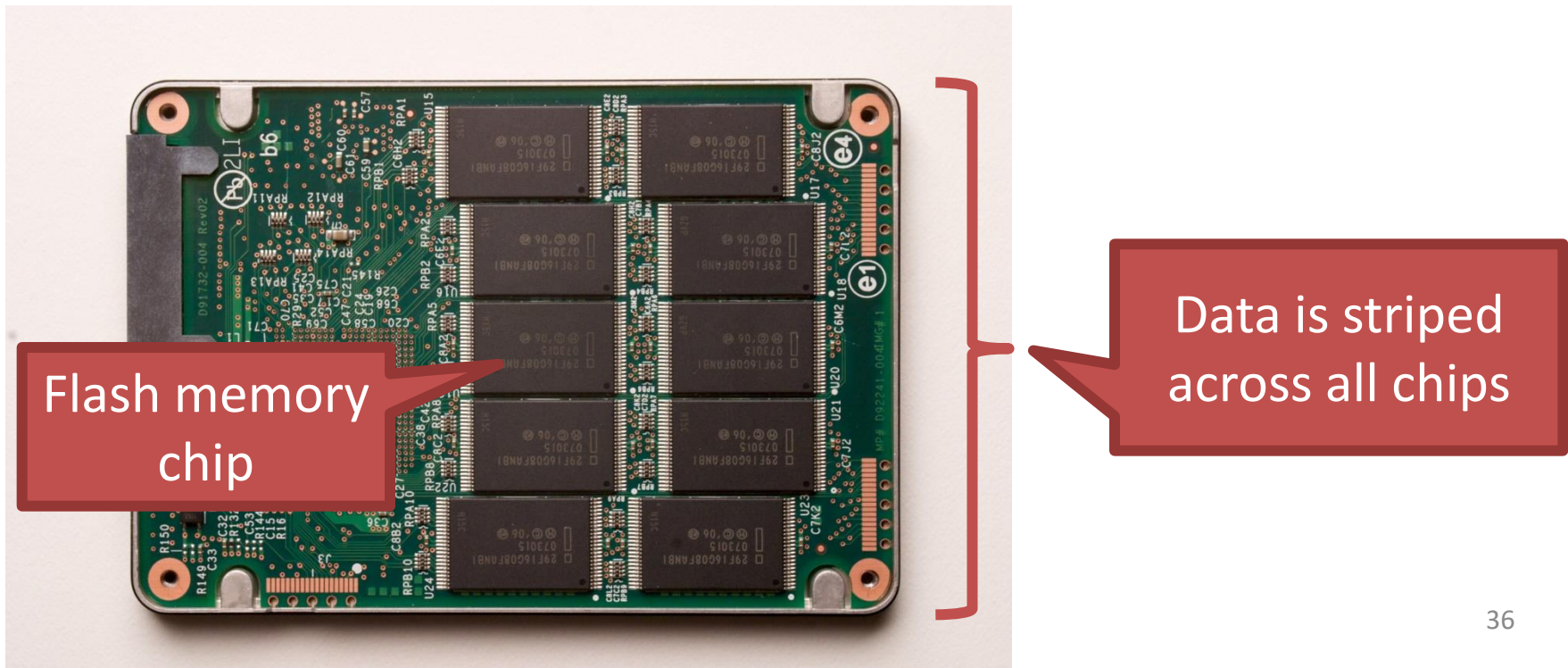
- Hard Drives
- SSD

Beyond Spinning Disks

- Hard drives have been around since 1956
 - The cheapest way to store large amounts of data
 - Sizes are still increasing rapidly
- However, hard drives are typically the slowest component in most computers
 - CPU and RAM operate at GHz
 - PCI-X and Ethernet are GB/s
- Hard drives are not suitable for mobile devices
 - Fragile mechanical components can break
 - The disk motor is extremely power hungry

Solid State Drives

- NAND flash memory-based drives
 - High voltage is able to change the configuration of a floating-gate transistor
 - State of the transistor interpreted as binary data



Advantages of SSDs

- More resilient against physical damage
 - No sensitive read head or moving parts
 - Immune to changes in temperature
- Greatly reduced power consumption
 - No mechanical, moving parts
- Much faster than hard drives
 - >500 MB/s vs ~200 MB/s for hard drives
 - No penalty for random access
 - Each flash cell can be addressed directly
 - No need to rotate or seek
 - Extremely high throughput
 - Although each flash chip is slow, they are RAIDed

Challenges with Flash

- Flash memory is written in pages, but erased in blocks
 - Pages: 4 – 16 KB, Blocks: 128 – 256 KB
 - Thus, flash memory can become fragmented
- Flash memory can only be written a fixed number of times
 - Typically 3000 – 5000 cycles for MLC
 - SSDs use [wear leveling](#) to evenly distribute writes across all flash cells

Write Amplification

G moved to new block by the garbage collector

Cleaned block can now be rewritten

Block X			
K	D	G	C'
L	E	A'	D'
C	F	B'	E'

Block Y			
G	C''	F''	J
A''	D''	H	A'''
B''	E''	I	B'''

- Once all pages have been written, valid pages must be consolidated to free up space
- **Write amplification**: a write triggers garbage collection/compaction
 - One or more blocks must be read, erased, and rewritten before the write can proceed

Garbage Collection

- Garbage collection (GC) is vital for the performance of SSDs
- Older SSDs had fast writes up until all pages were written once
 - Even if the drive has lots of “free space,” each write is amplified, thus reducing performance
- Many SSDs over-provision to help the GC
 - 240 GB SSDs actually have 256 GB of memory
- Modern SSDs implement background GC
 - However, this doesn't always work correctly

Wear Leveling

- Recall: each flash cell wears out after several thousand writes
- SSDs use **wear leveling** to spread writes across all cells
 - Typical consumer SSDs should last ~5 years

W

If the GC runs now, page G must be copied

Examples

Dynamic Wear Leveling

Wait as long as possible before garbage collecting

Block X			
K	D	G	C'
L	E	A'	D'
C	F	B'	E'

Block Y			
F'	C''	F''	G'
A''	D''	H	A'''
B''	E''	I	B'''

Static Wear Leveling

Blocks with long lived data receive less wear

Block X			
M*	D	G	J
N*	E	H	K
O*	F	I	L

Block Y			
A	D	G	J
B	E	H	K
C	F	I	L

SSD controller periodically swap long lived data to different blocks

SSD Controllers

- SSDs are extremely complicated internally
- All operations handled by the SSD controller
 - Maps LBAs to physical pages
 - Keeps track of free pages, controls the GC
 - May implement background GC
 - Performs wear leveling via data rotation
- Controller performance is crucial for overall SSD performance



Flavors of NAND Flash Memory

Multi-Level Cell (MLC)

- Multiple bits per flash cell
 - For two-level: 00, 01, 10, 11
 - 2, 3, and 4-bit MLC is available
- Higher capacity and cheaper than SLC flash
- Lower throughput due to the need for error correction
- 3000 – 5000 write cycles
- Consumes more power

Consumer-grade drives

Single-Level Cell (SLC)

- One bit per flash cell
 - 0 or 1
- Lower capacity and more expensive than MLC flash
- Higher throughput than MLC
- 10000 – 100000 write cycles

Expensive, enterprise drives

Feature	HDD	SSD
Storage Technology	spinning magnetic disks	NAND flash memory
Speed	Slower boot times and file access.	Much faster boot times and file access.
Durability	prone to damage.	More durable
Noise	noise due to spinning disks.	Operates silently.
Power Consumption	Consumes more power.	Uses less power

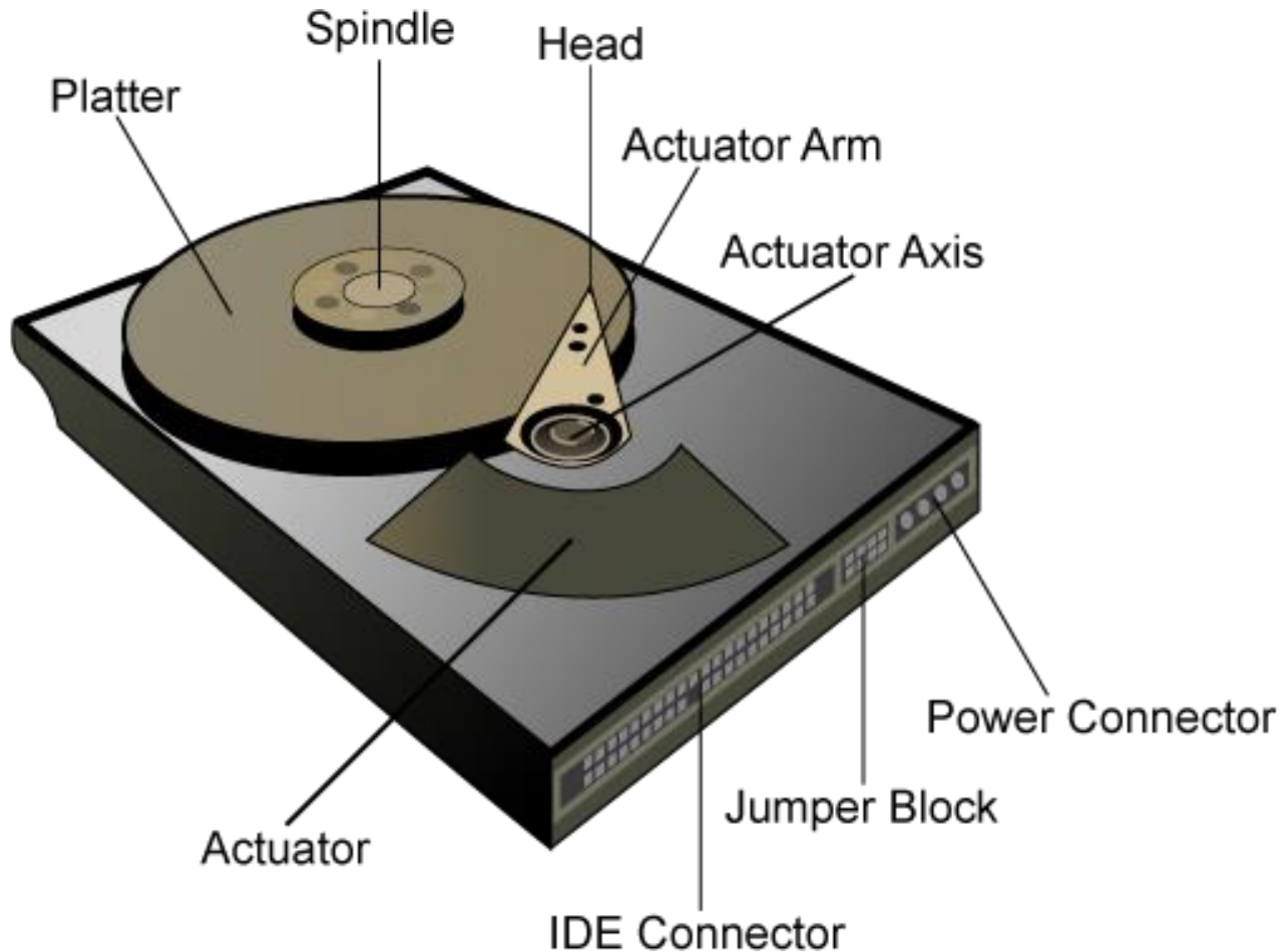
Feature	HDD	SSD
Cost	More affordable per GB, cost-effective	More expensive per GB but offers better performance.
Best Use Case	Ideal for budget-friendly, high-capacity storage.	Best for high-speed computing, fast system performance.
Reliability	Less reliable due to moving parts (e.g., head crash)	More reliable (no moving parts, less risk of failure)
Lifespan	Wears out over time due to mechanical movement.	Lasts longer but has limited write cycles.

Storage Devices

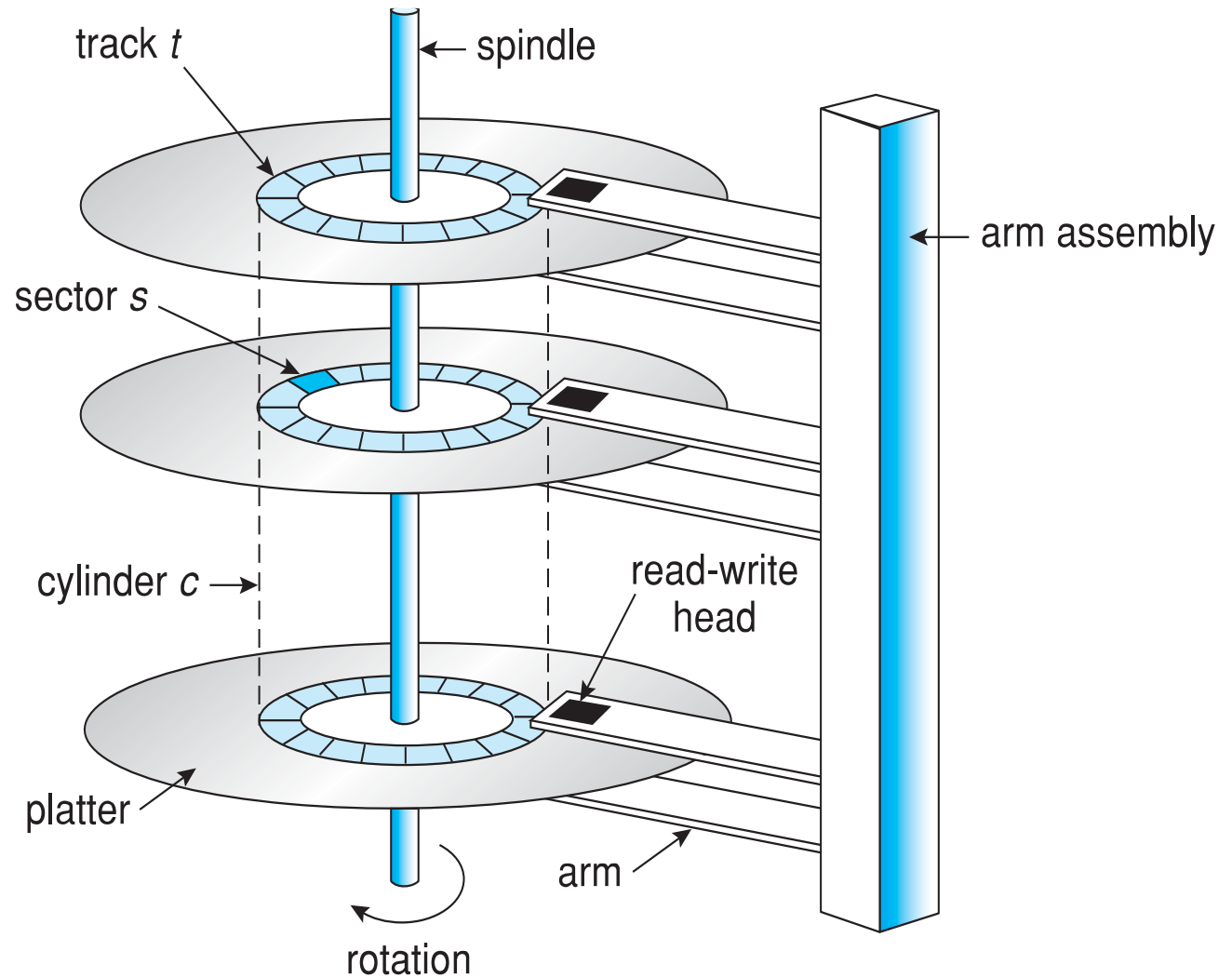
Hard Drives

SSD

Hard Drive Hardware



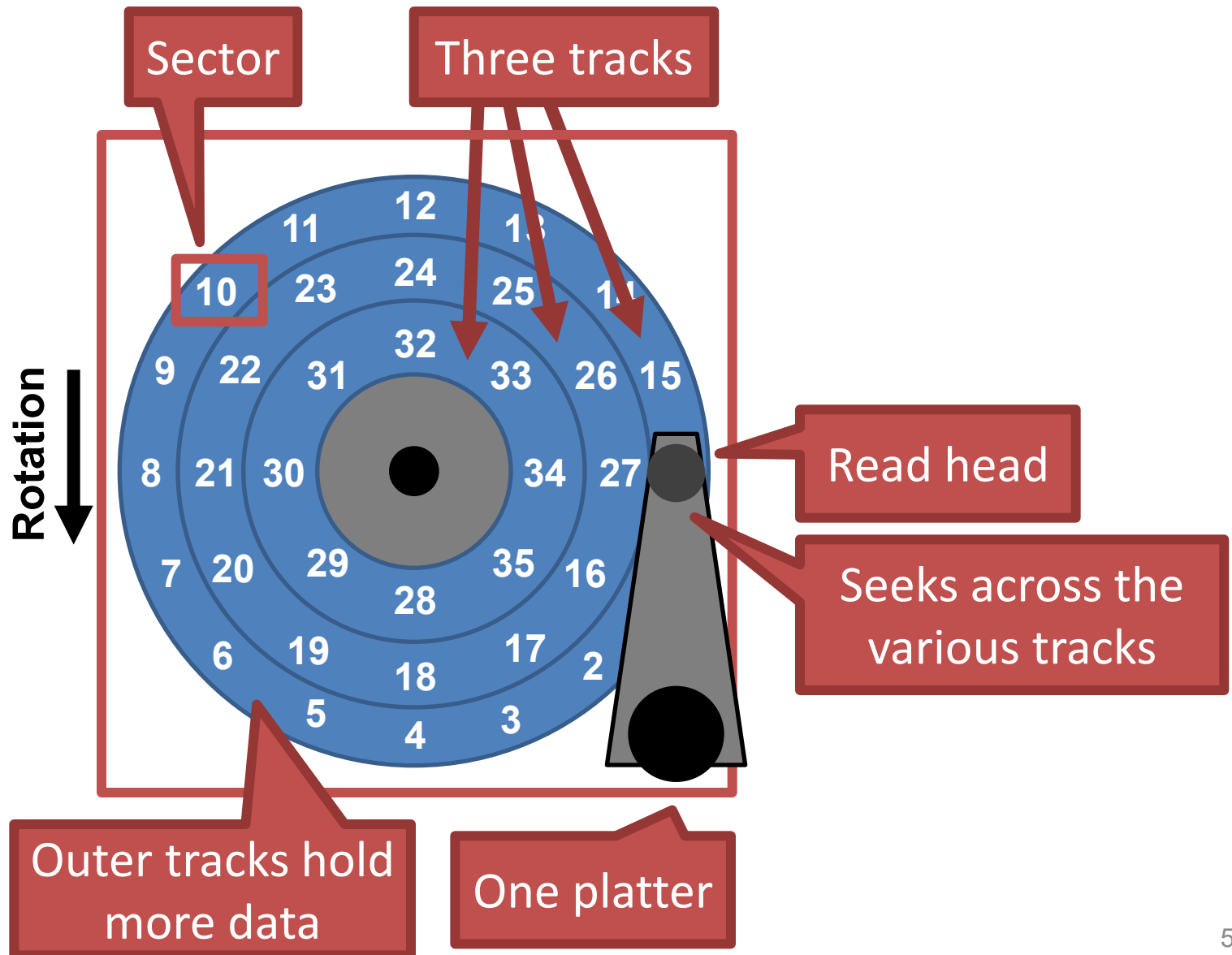
A Multi-Platter Disk



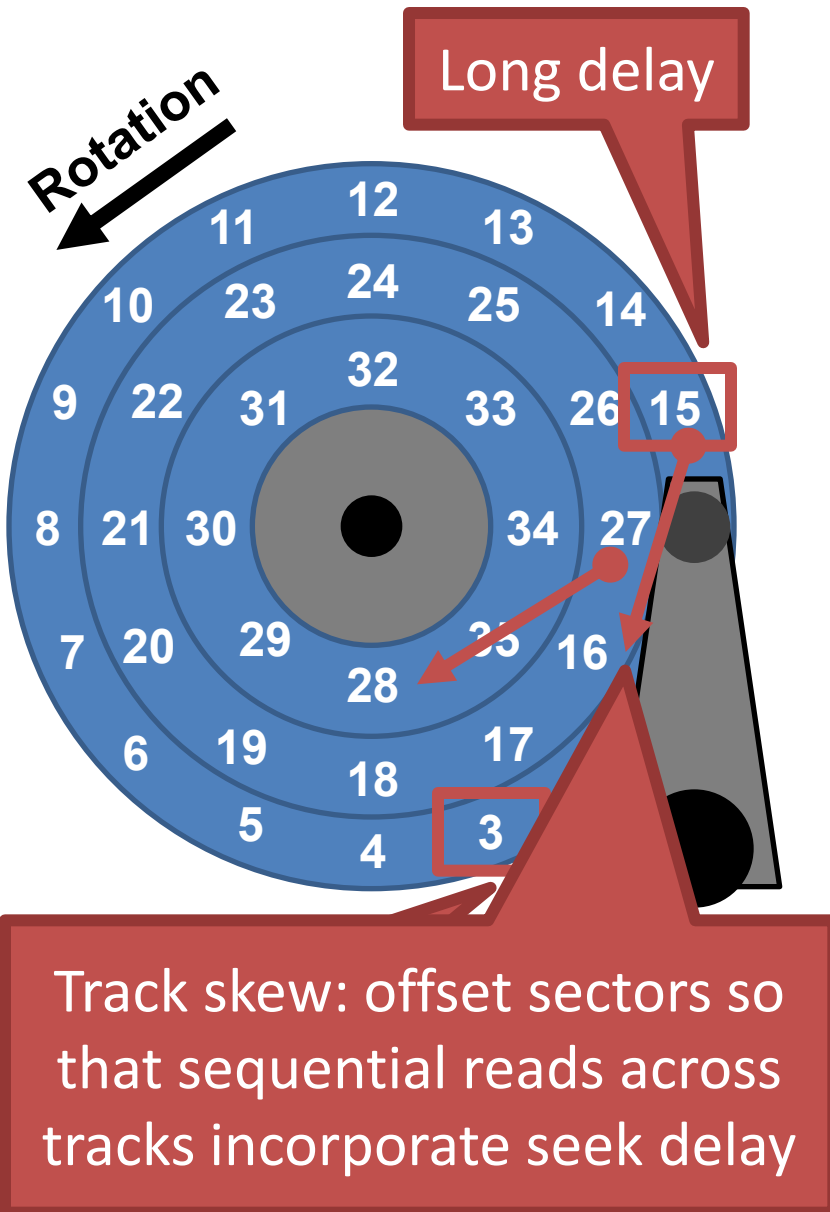
Addressing and Geometry

- Externally, hard drives expose a large number of **sectors** (blocks)
 - Typically, 512 or 4096 bytes
 - Individual sector writes are **atomic**
- Drive geometry
 - Sectors arranged into **tracks**
 - A **cylinder** is a particular track on multiple platters
 - Tracks arranged in concentric circles on **platters**
 - A disk may have multiple, double-sided platters
- Drive motor spins the platters at a constant rate
 - Measured in revolutions per minute (RPM)

Geometry Example



Types of Delay With Disks



Three types of delay

1. Rotational Delay
 - Time to rotate the desired sector to the read head
 - Related to RPM
2. Seek delay
 - Time to move the read head to a different track
3. Transfer time
 - Time to read or write bytes

How To Calculate Transfer Time



	Cheetah 15K.5	Barracuda
Capacity	300 GB	1 TB
RPM	15000	7200
Avg. Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s

Transfer time

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

Assume we are transferring 4096 bytes

Cheetah

Time for one rotation = 60 Sec / 15000

$$= 60 \times 1000 \text{ Mili second} / 15000$$

$$= 4.000 \text{ ms}$$

Avg rotation time is = $4.000 \text{ ms} / 2 = 2 \text{ ms}$

Time to transfer 1 byte = $1 \text{ sec} / 125 \times 2^{20}$

$$= 1000 \text{ ms} / 125 \times 2^{20}$$

Time to Transfer 4096 byte = $4096 \times 1000 \text{ ms} / 125 \times 2^{20}$

$$= 0.03125 \text{ ms}$$

Total time = $4 \text{ ms} + 2 \text{ ms} + 0.03125 \text{ ms} = 6.03125 \text{ ms}$.

Step 1: Time for one rotation

$$\begin{aligned} T &= \frac{60 \text{ sec}}{7200} \\ &= \frac{60 \times 1000 \text{ ms}}{7200} \\ &= 8.333 \text{ ms (one rotation)} \end{aligned}$$

Step 2: Average rotational latency

$$= \frac{8.333}{2} = 4.167 \text{ ms}$$

Step 3: Transfer time for 1 byte

$$\begin{aligned} t &= \frac{1 \text{ sec}}{105 \times 2^{20}} \\ &= \frac{1000 \text{ ms}}{105 \times 2^{20}} \end{aligned}$$

Step 4: Transfer time for 4096 bytes (4 KB)

$$\begin{aligned} t &= \frac{4096 \times 1000}{105 \times 2^{20}} \\ &= 0.0372 \text{ ms (approx)} \end{aligned}$$

Step 5: Total time (Avg Seek + Avg Rot + Transfer)

$$\begin{aligned} &= 9 \text{ ms} + 4.167 \text{ ms} + 0.0372 \text{ ms} \\ &= 13.204 \text{ ms (approx)} \end{aligned}$$

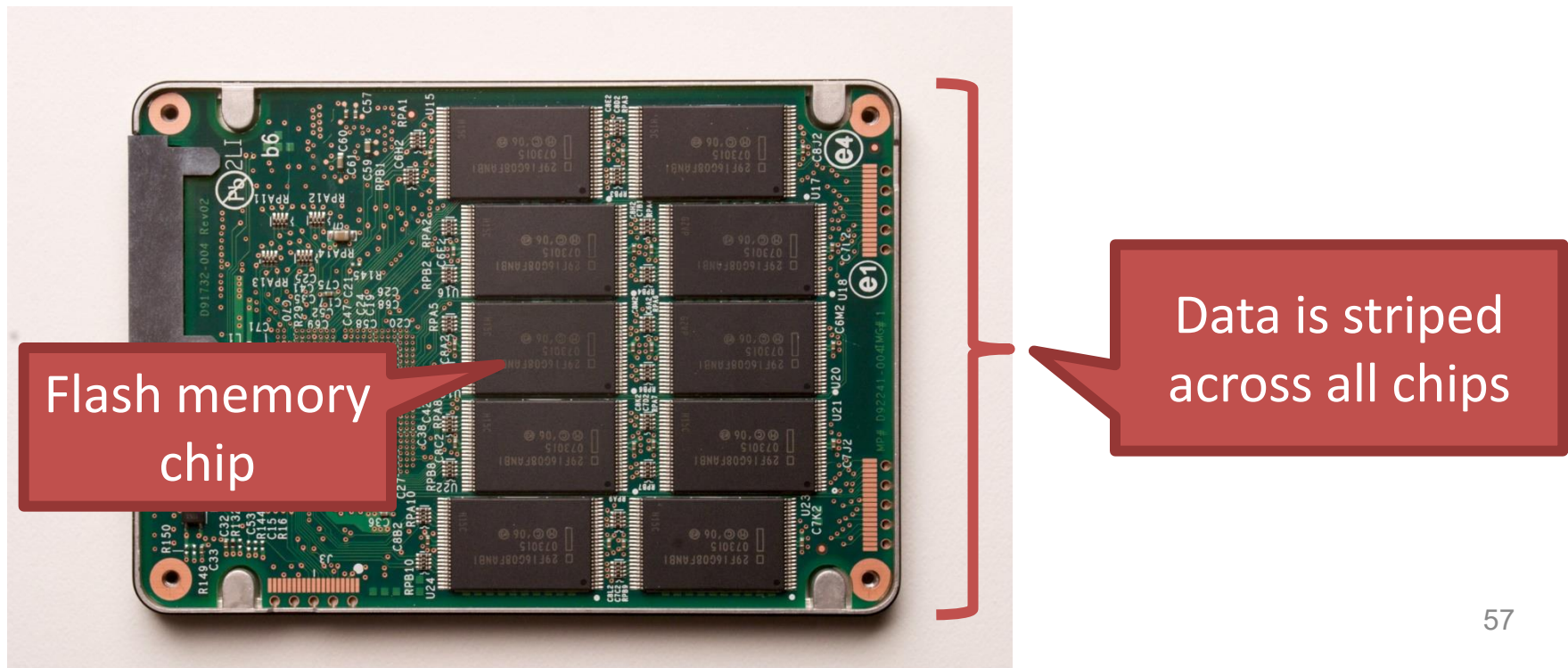
- Hard Drives
- SSD

Beyond Spinning Disks

- Hard drives have been around since 1956
 - The cheapest way to store large amounts of data
 - Sizes are still increasing rapidly
- However, hard drives are typically the slowest component in most computers
 - CPU and RAM operate at GHz
 - PCI-X and Ethernet are GB/s
- Hard drives are not suitable for mobile devices
 - Fragile mechanical components can break
 - The disk motor is extremely power hungry

Solid State Drives

- NAND flash memory-based drives
 - High voltage is able to change the configuration of a floating-gate transistor
 - State of the transistor interpreted as binary data



Advantages of SSDs

- More resilient against physical damage
 - No sensitive read head or moving parts
 - Immune to changes in temperature
- Greatly reduced power consumption
 - No mechanical, moving parts
- Much faster than hard drives
 - >500 MB/s vs ~200 MB/s for hard drives
 - No penalty for random access
 - Each flash cell can be addressed directly
 - No need to rotate or seek
 - Extremely high throughput
 - Although each flash chip is slow, they are RAIDed

Challenges with Flash

- Flash memory is written in pages, but erased in blocks
 - Pages: 4 – 16 KB, Blocks: 128 – 256 KB
 - Thus, flash memory can become fragmented
- Flash memory can only be written a fixed number of times
 - Typically 3000 – 5000 cycles for MLC
 - SSDs use [wear leveling](#) to evenly distribute writes across all flash cells

Write Amplification

G moved to new block by the garbage collector

Cleaned block can now be rewritten

Block X			
K	D	G	C'
L	E	A'	D'
C	F	B'	E'

Block Y			
G	C''	F''	J
A''	D''	H	A'''
B''	E''	I	B'''

- Once all pages have been written, valid pages must be consolidated to free up space
- **Write amplification**: a write triggers garbage collection/compaction
 - One or more blocks must be read, erased, and rewritten before the write can proceed

Garbage Collection

- Garbage collection (GC) is vital for the performance of SSDs
- Older SSDs had fast writes up until all pages were written once
 - Even if the drive has lots of “free space,” each write is amplified, thus reducing performance
- Many SSDs over-provision to help the GC
 - 240 GB SSDs actually have 256 GB of memory
- Modern SSDs implement background GC
 - However, this doesn't always work correctly

Wear Leveling

- Recall: each flash cell wears out after several thousand writes
- SSDs use wear leveling to spread writes across all cells
 - Typical consumer SSDs should last ~5 years

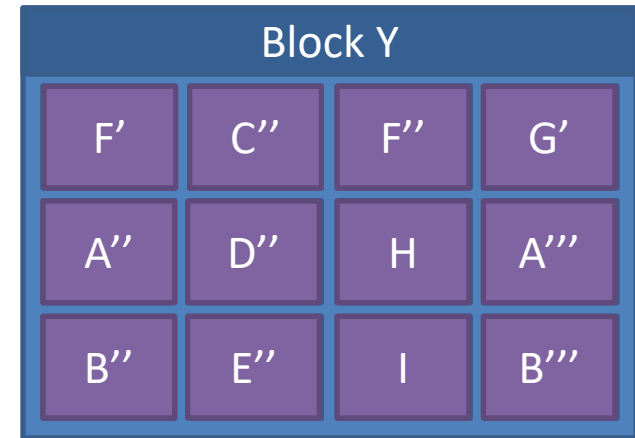
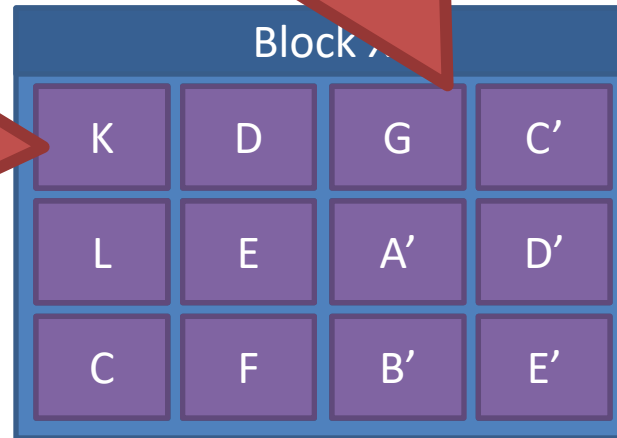
W

If the GC runs now, page G must be copied

Examples

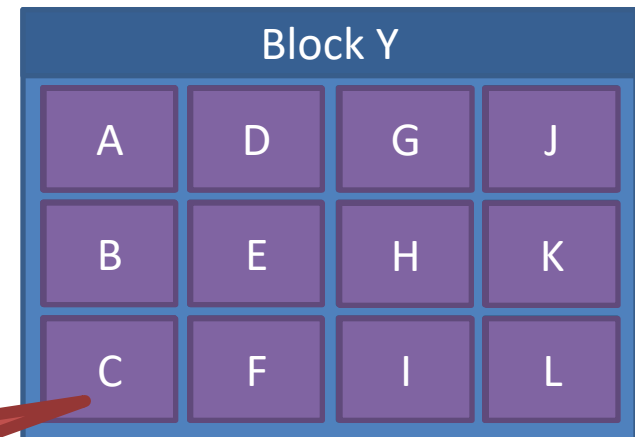
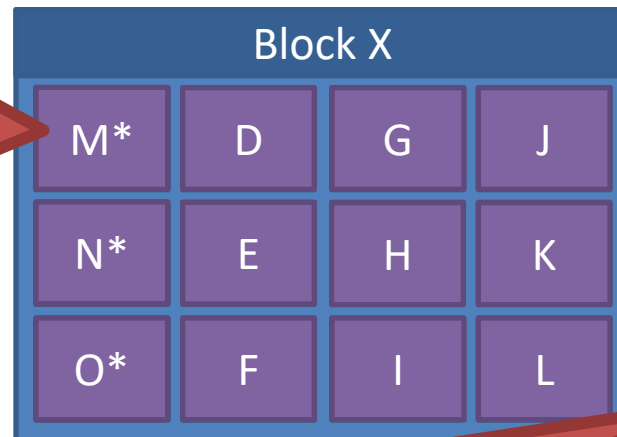
Dynamic Wear Leveling

Wait as long as possible before garbage collecting



Static Wear Leveling

Blocks with long lived data receive less wear



SSD controller periodically swap long lived data to different blocks

SSD Controllers

- SSDs are extremely complicated internally
- All operations handled by the SSD controller
 - Maps LBAs to physical pages
 - Keeps track of free pages, controls the GC
 - May implement background GC
 - Performs wear leveling via data rotation
- Controller performance is crucial for overall SSD performance



Flavors of NAND Flash Memory

Multi-Level Cell (MLC)

- Multiple bits per flash cell
 - For two-level: 00, 01, 10, 11
 - 2, 3, and 4-bit MLC is available
- Higher capacity and cheaper than SLC flash
- Lower throughput due to the need for error correction
- 3000 – 5000 write cycles
- Consumes more power

Consumer-grade drives

Single-Level Cell (SLC)

- One bit per flash cell
 - 0 or 1
- Lower capacity and more expensive than MLC flash
- Higher throughput than MLC
- 10000 – 100000 write cycles

Expensive, enterprise drives

Feature	HDD	SSD
Storage Technology	spinning magnetic disks	NAND flash memory
Speed	Slower boot times and file access.	Much faster boot times and file access.
Durability	prone to damage.	More durable
Noise	noise due to spinning disks.	Operates silently.
Power Consumption	Consumes more power.	Uses less power

Feature	HDD	SSD
Cost	More affordable per GB, cost-effective	More expensive per GB but offers better performance.
Best Use Case	Ideal for budget-friendly, high-capacity storage.	Best for high-speed computing, fast system performance.
Reliability	Less reliable due to moving parts (e.g., head crash)	More reliable (no moving parts, less risk of failure)
Lifespan	Wears out over time due to mechanical movement.	Lasts longer but has limited write cycles.

Que: A computer with an HDD takes an average of 45 seconds to boot, while a computer with an SSD takes only 10 seconds.

Problem a: If a user with an HDD-equipped computer starts their system 3 times a day for a year, how many hours of boot-up time would they save by upgrading to an SSD?

Problem b: What is the percentage improvement in boot time when switching from the HDD to the SSD?

Solution a

Step 1: Boot time per start

- HDD boot = 45 seconds
- SSD boot = 10 seconds
- **Time saved per boot = $45 - 10 = 35$ seconds**

Step 2: Boots per year

- Starts per day = 3
- Days per year = 365
- Total starts = $3 \times 365 = 1095$

Step 3: Total time saved per year

$$\begin{aligned}\text{Time saved} &= 1095 \times 35 \text{ seconds} \\ &= 38325 \text{ seconds}\end{aligned}$$

Step 4: Convert to hours

$$38325 \div 3600 \approx 10.65 \text{ hours}$$

Solution b

Percentage improvement

Formula:

$$\text{Improvement \%} = \frac{\text{Old time} - \text{New time}}{\text{Old time}} \times 100$$

$$= \frac{45 - 10}{45} \times 100$$

$$= \frac{35}{45} \times 100 \approx 77.78\%$$

The improvement is about **77.8%**.

Booting

Disk Partitioning

- A logical division of a hard drive (HDD) or solid-state drive (SSD) that allows it to be managed as separate storage units.
- Organize data and files.
- Allow installation of multiple operating systems.
- Improve performance and data management.
- The two primary schemes are MBR and GPT

MBR

- Stored at the first sector of a storage device.
- Contains:
 - Boot Loader (Executable code that initiates the operating system).
 - Partition Table: A record of the disk's partitions.

MBR Limitations

- Partition Limit: Supports a maximum of four primary partitions.
- Disk Size Limit: Cannot address disks larger than 2 terabytes (TB).
- Reliability: The MBR partition table is stored in a single, unbacked-up location. If this data is corrupted, the disk may become unbootable.
- Compatible with BIOS.

GPT

- A modern and robust partitioning standard that was developed as part of the Unified Extensible Firmware Interface (UEFI).
- Stores a primary GPT header at the start of the disk and a backup header at the end for redundancy.
- Compatible with UEFI.

GPT Advantages

- Increased Partition Limit: Supports up to 128 primary partitions in Windows.
- Can manage disks up to 9.4 zettabytes (ZB).
- Improved Reliability.

Volume Boot Record

- Definition: The VBR is the first sector of an individual hard disk or solid-state drive (SSD) partition that contains an operating system.
- Purpose: The VBR contains a small program (the Volume Boot Code) specific to the file system (e.g., NTFS, FAT32) and operating system of that partition.
- Key Function: Its primary job is to load the rest of the operating system's boot files.

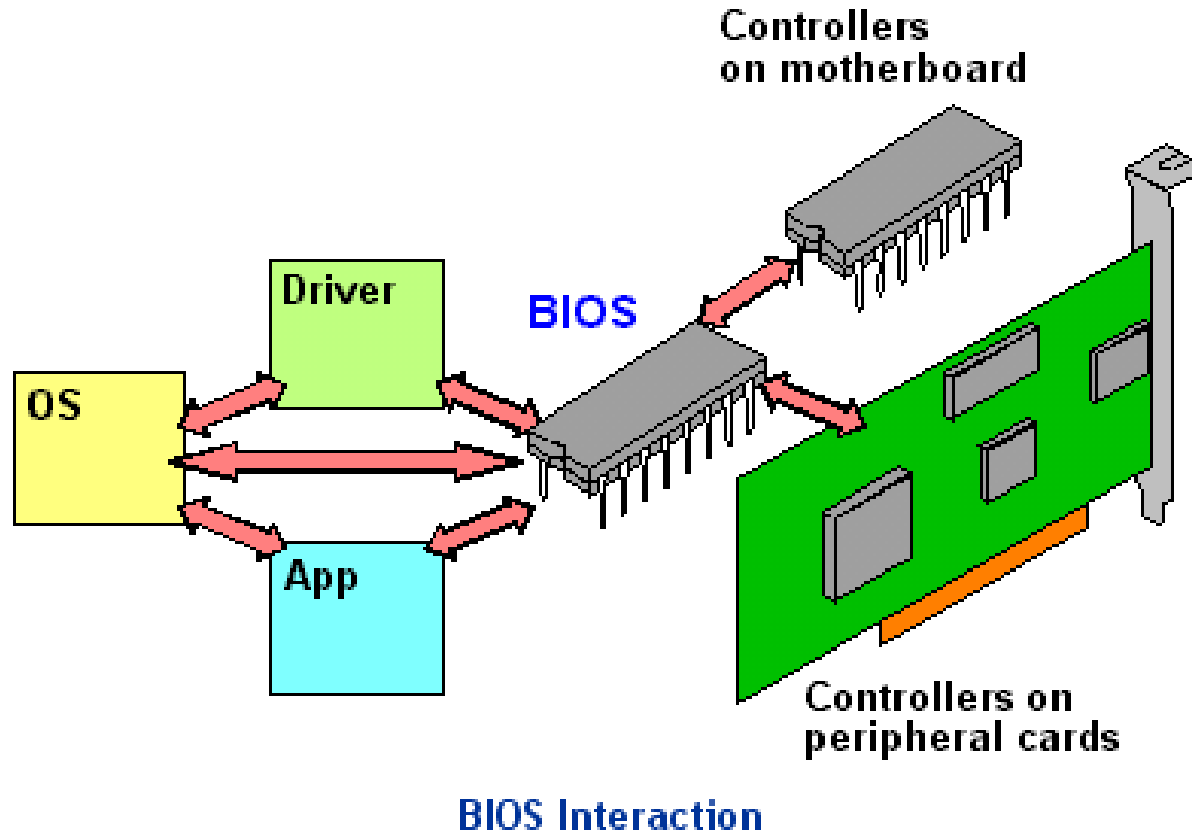
Why is Booting Required ?

- Hardware doesn't know where the operating system resides and how to load it.
- Need a special program to do this job – **Bootstrap** loader.
 - E.g. BIOS – Boot Input Output System.
- Bootstrap loader locates the kernel, loads it into main memory and starts its execution.
- In some systems, a simple bootstrap loader fetches a more complex boot program from disk, which in turn loads the kernel.

How Boot process occurs ?

- Reset event on CPU (power up, reboot) causes instruction register to be loaded with a predefined memory location. It contains a jump instruction that transfers execution to the location of Bootstrap program.
- This program is form of ROM, since RAM is in unknown state at system startup. ROM is convenient as it needs no initialization and can't be affected by virus.

BIOS Interaction



Tasks performed at boot up

- Run diagnostics to determine the state of machine. If diagnostics pass, booting continues.
- Runs a [Power-On Self Test](#) (*POST*) to check the devices that the computer will rely on, are functioning.
- BIOS goes through a preconfigured list of devices until it finds one that is bootable. If it finds no such device, an error is given and the boot process stops.
- Initializes CPU registers, device controllers and contents of the main memory. After this, it loads the OS.

Tasks performed at boot up (Contd)

- On finding a bootable device, the BIOS loads and executes its [boot sector](#). In the case of a hard drive, this is referred to as the [master boot record](#) (*MBR*) and is often not OS specific.
- The MBR code checks the [partition table](#) for an active partition. If one is found, the MBR code loads that partition's [boot sector](#) and executes it.
- The boot sector is often [operating system](#) specific, however in most operating systems its main function is to load and execute a [kernel](#), which continues startup.

Example : DOS

- After identifying the location of boot files, BIOS looks at the first sector (512 bytes) and copies information to specific location in RAM (7C00H) - [Boot Record](#).
- Control passes from BIOS to a program residing in the boot record.
- Boot record loads the initial system file into RAM. For DOS, it is IO.SYS .
- The initial file, IO.SYS includes a file called SYSINIT which loads the remaining OS into the RAM.
- SYSINIT loads a system file MSDOS.SYS that knows how to work with BIOS.
- One of the first OS files that is loaded is the system configuration file, CONFIG.SYS in case of DOS. Information in the configuration file tells loading program which OS files need to be loaded (e.g. drivers)
- Another special file that is loaded is one which tells what specific applications or commands user wants to be performed as part of booting process. In DOS, it is AUTOEXEC.BAT. In Windows, it's WIN.INI .

Operating Systems

Introduction to
Operating System (OS)

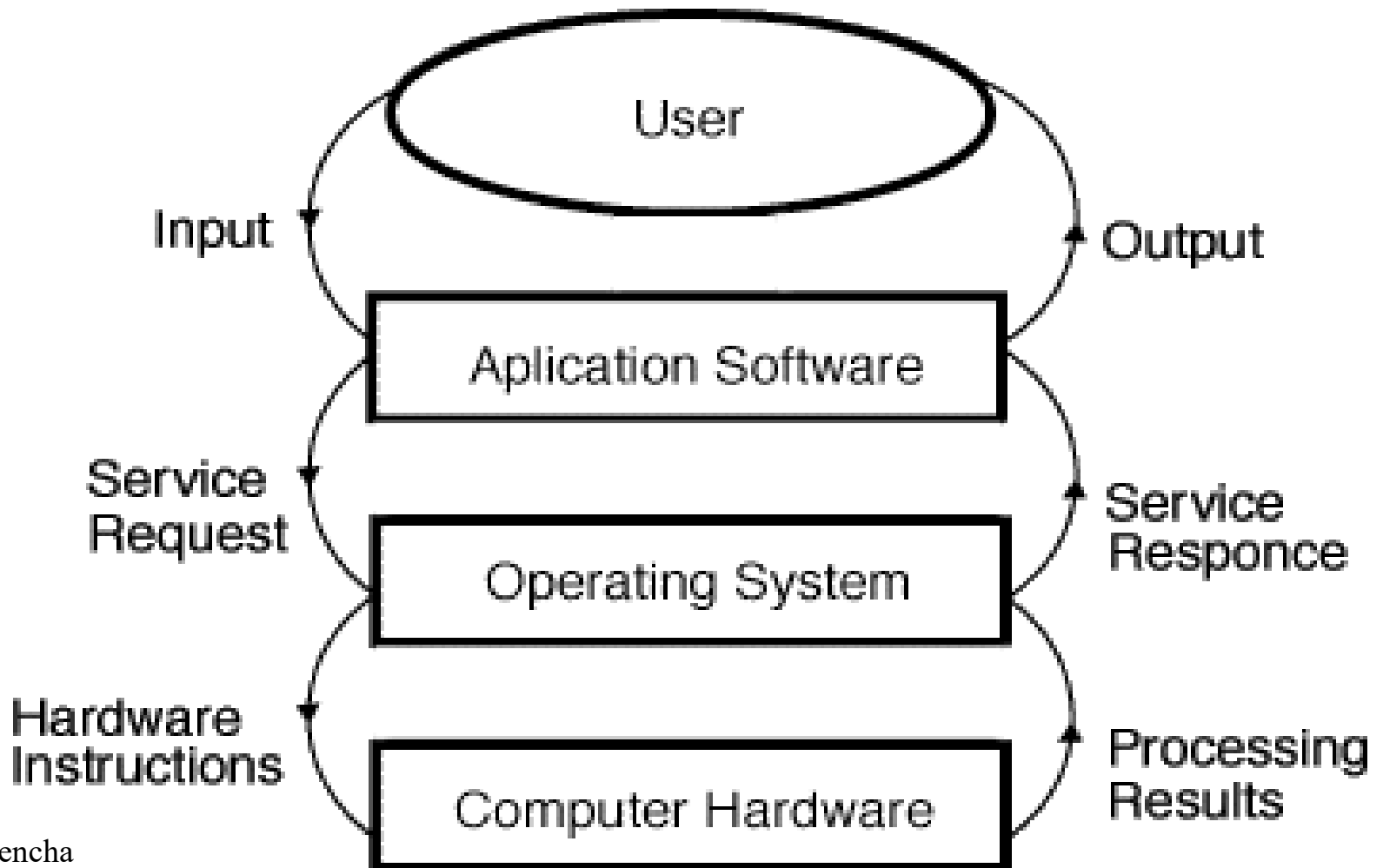
What is an Operating System (2)?

- An Operating System is a program that acts as an intermediary/interface between a user of a computer and the computer hardware.
- OS goals:
 - Control/execute user/application programs.
 - Make the computer system convenient to use.
 - Ease the solving of user problems.
 - Use the computer hardware in an efficient manner.

Services provided by an OS

- Facilities for program creation
 - editors, compilers, linkers, debuggers, etc.
- Program execution
 - loading in memory, I/O and file initialization.
- Access to I/O and files
 - deals with the specifics of I/O and file formats.
- System access
 - resolves conflicts for resource contention.
 - protection in access to resources and data.

Dynamic View of System Components



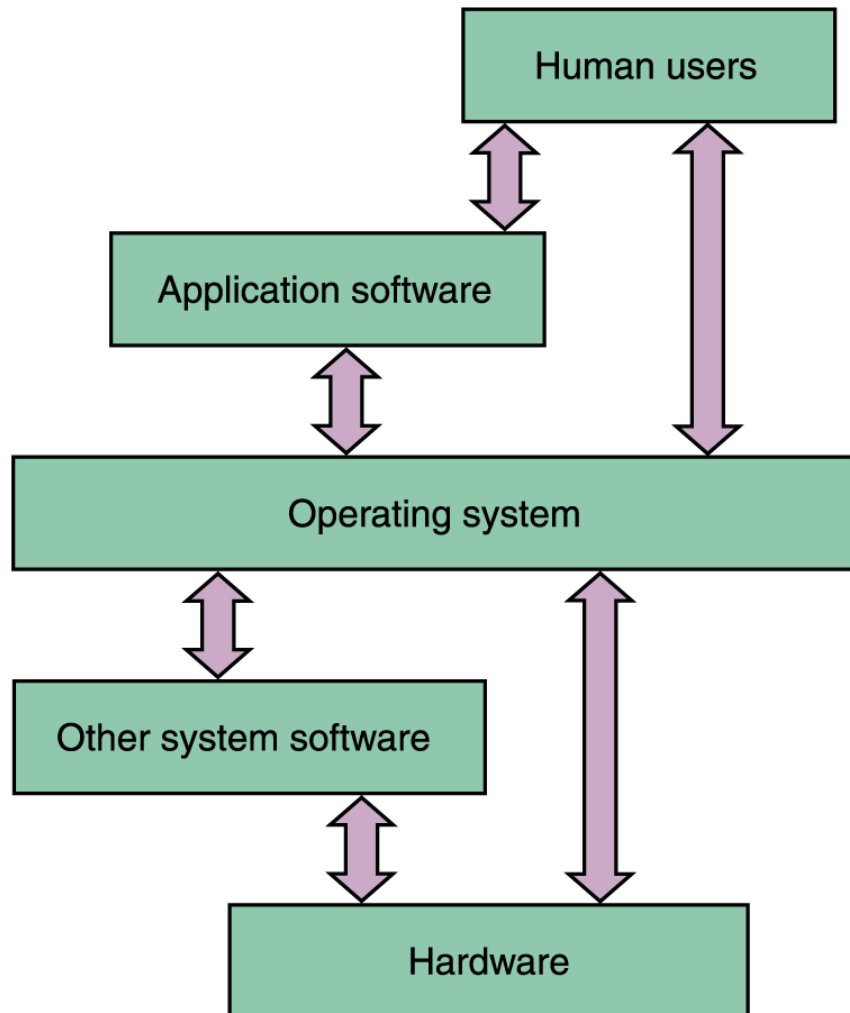
Roles of an Operating System

Operating system

System software that

- **manages** computer resources, such as memory and input/output devices
- **provides** an interface through which a human can interact with the computer
- **allows** an application program to interact with these other system resources

Roles of an Operating System



What operating systems have you used?

Figure 10.1
An operating system interacts with many aspects of a computer system.

Roles of an Operating System

The various roles of an operating system generally revolve around the idea of “sharing nicely”

An operating system manages resources, and these resources are often shared in one way or another among programs that want to use them

Resource Management

Multiprogramming

The technique of keeping multiple programs that compete for access to the CPU in main memory at the same time so that they can execute

Memory management

The process of keeping track of what programs are in memory and where in memory they reside

Resource Management

Process

A program in execution

Process management

The act of carefully tracking the progress of a process and all of its intermediate states

CPU scheduling

Determining which process in memory is executed by the CPU at any given point

Evolution of OS:

1. Mainframe Systems

Reduce setup time by batching similar jobs Automatic job sequencing – automatically transfers control from one job to another. First rudimentary

operating system. Resident monitor

- initial control in monitor
- control transfers to job
- when job completes control transfers back to monitor

Evolution of OS:

2. Batch Processing Operating System:

- This type of OS accepts more than one jobs and these jobs are batched/ grouped together according to their similar requirements. This is done by computer operator. Whenever the computer becomes available, the batched jobs are sent for execution and gradually the output is sent back to the user.
- It allowed only one program at a time.
- This OS is responsible for scheduling the jobs according to priority and the resource required.

Evolution of OS:

3. Multiprogramming Operating System:

- This type of OS is used to execute more than one jobs simultaneously by a single processor. it increases CPU utilization by organizing jobs so that the CPU always has one job to execute.

The concept of multiprogramming is described as follows:

- All the jobs that enter the system are stored in the job pool(in disc). The operating system loads a set of jobs from job pool into main memory and begins to execute.
- During execution, the job may have to wait for some task, such as an I/O operation, to complete. In a multiprogramming system, the operating system simply switches to another job and executes.

Evolution of OS:

Contd.

- When that job needs to wait, the CPU is switched to another job, and so on.
- When the first job finishes waiting and it gets the CPU back.
- As long as at least one job needs to execute, the CPU is never idle.
- Multiprogramming operating systems use the mechanism of job scheduling and CPU scheduling.

Evolution of OS:

4. Time-Sharing/multitasking Operating Systems

Time sharing (or multitasking) OS is a logical extension of multiprogramming. It provides extra facilities such as:

- Faster switching between multiple jobs to make processing faster.
- Allows multiple users to share computer system simultaneously.
- The users can interact with each job while it is running.

These systems use a concept of virtual memory for effective utilization of memory space. Hence, in this OS, no jobs are discarded. Each one is executed using virtual memory concept. It uses CPU scheduling, memory management, disc management and security management.

Evolution of OS:

5. Multiprocessor Operating Systems

Multiprocessor operating systems are also known as parallel OS or tightly coupled OS. Such operating systems have more than one processor in close communication that sharing the computer bus, the clock and sometimes memory and peripheral devices. It executes multiple jobs at same time and makes the processing faster.

Multiprocessor systems have three main advantages:

- Increased throughput: By increasing the number of processors,
- Economy of scale: Multiprocessor systems can save more money than multiple single-processor systems
- Increased reliability: If one processor fails to done its task, then each of the remaining processors must pick up a share of the work of the failed processor.

Evolution of OS:

5. Distributed Operating Systems

In distributed system, the different machines are connected in a network and each machine has its own processor and own local memory. In this system, the operating systems on all the machines work together to manage the collective network resource.

Advantages of distributed systems.

- Resources Sharing
- Computation speed up – load sharing
- Reliability
- Communications
- Requires networking infrastructure.
- Local area networks (LAN) or Wide area networks (WAN)

Module 4: Processes

- Process Concept
- Process Scheduling
- Operation on Processes
- Cooperating Processes
- Interprocess Communication

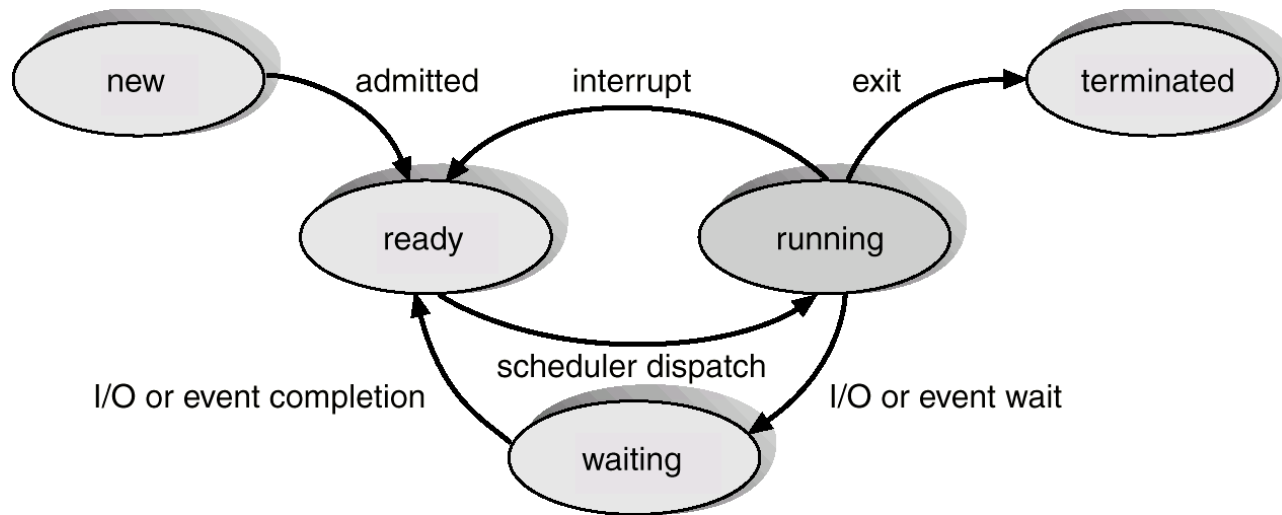
Process Concept

- An operating system executes a variety of programs:
 - Batch system – jobs
 - Time-shared systems – user programs or tasks
- Textbook uses the terms *job* and *process* almost interchangeably.
- Process – a program in execution; process execution must progress in sequential fashion.
- A process includes:
 - program counter
 - stack

Process State

- As a process executes, it changes *state*
 - new: The process is being created.
 - running: Instructions are being executed.
 - waiting: The process is waiting for some event to occur.
 - ready: The process is waiting to be assigned to a process.
 - terminated: The process has finished execution.

Diagram of Process State



Process Control Block (PCB)

pointer	process state
process number	
program counter	
registers	
memory limits	
list of open files	
• • •	

Process Scheduling Queues

- Job queue – set of all processes in the system.
- Ready queue – set of all processes residing in main memory, ready and waiting to execute.
- Device queues – set of processes waiting for an I/O device.
- Process migration between the various queues.

Schedulers

- Long-term scheduler (or job scheduler)
 - selects which processes should be brought into the ready queue.
- Short-term scheduler (or CPU scheduler) – selects which process should be executed next and allocates CPU.

Interprocess Communication (IPC)

- Mechanism for processes to communicate and to synchronize their actions.
- Message system – processes communicate with each other without resorting to shared variables.
- IPC facility provides two operations:
 - **send**(*message*) – message size fixed or variable
 - **receive**(*message*)
- If P and Q wish to communicate, they need to:
 - establish a *communication link* between them
 - exchange messages via send/receive
- Implementation of communication link
 - physical (e.g., shared memory, hardware bus)
 - logical (e.g., logical properties)

Memory Management

Operating systems must employ techniques to

- Track where and how a program resides in memory
- Convert **logical addresses** into actual **addresses**

Logical address

Reference to a stored value relative to the program making the reference

Physical address

Actual address in main memory

Memory Management

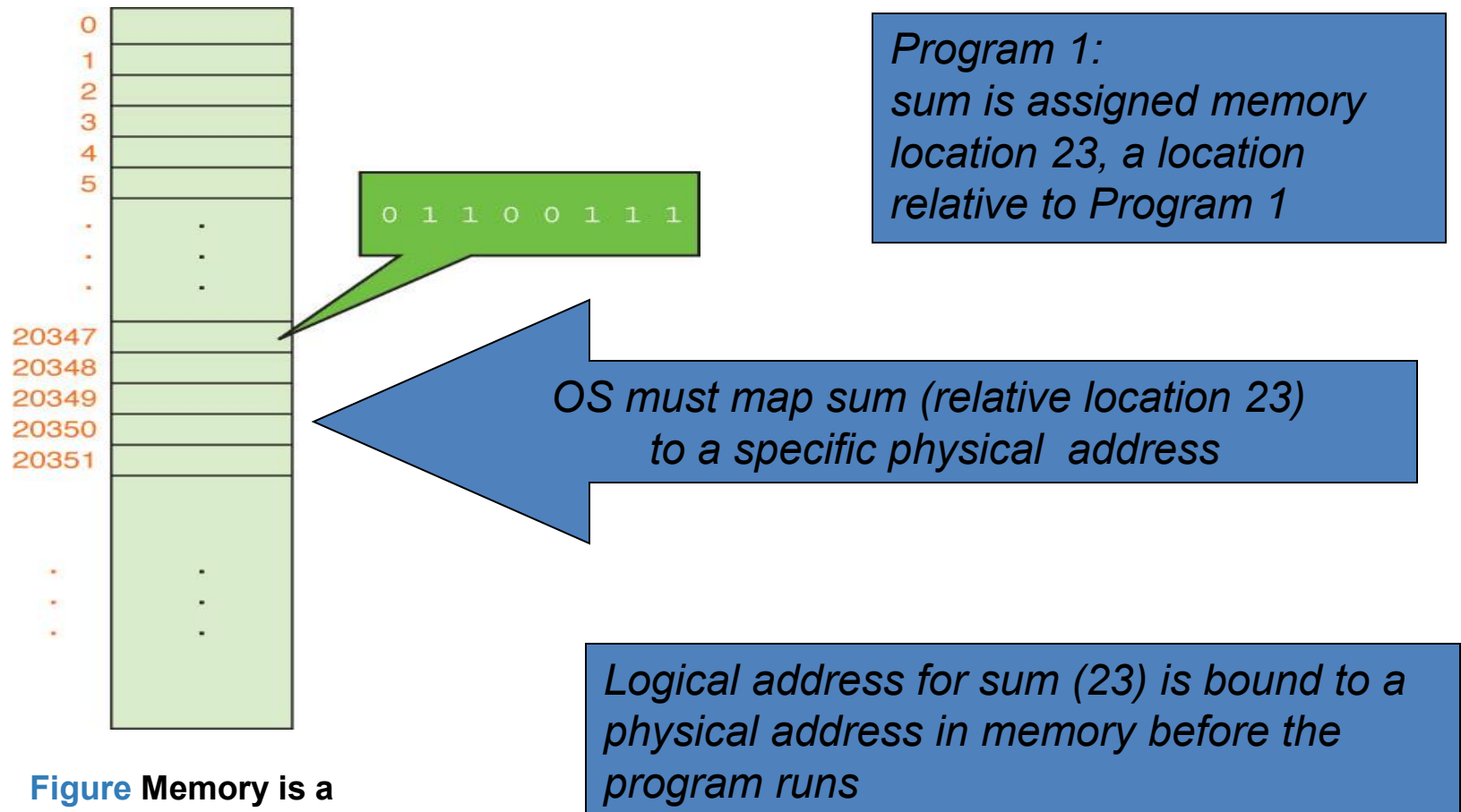
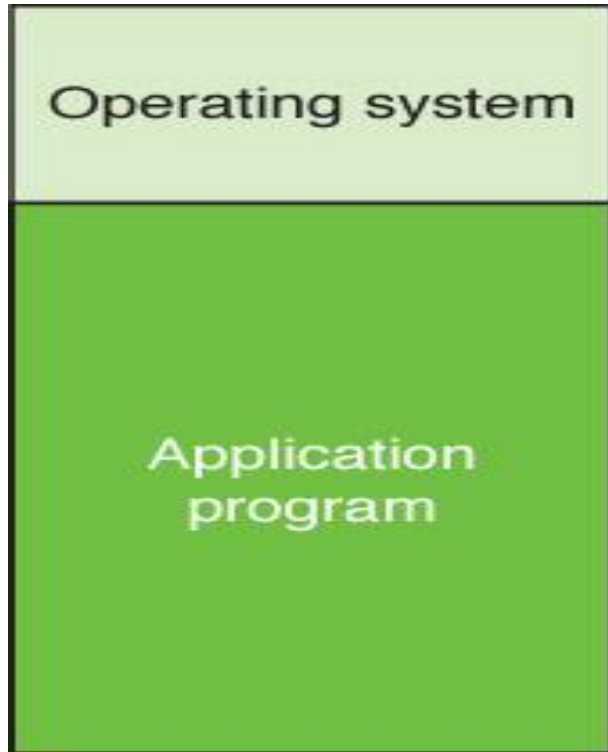


Figure Memory is a continuous set of bits referenced by specific addresses

Single Contiguous MM



There are only two programs in memory

The operating system

The application program

This approach is called **single contiguous memory management**

Figure Main memory divided into two sections

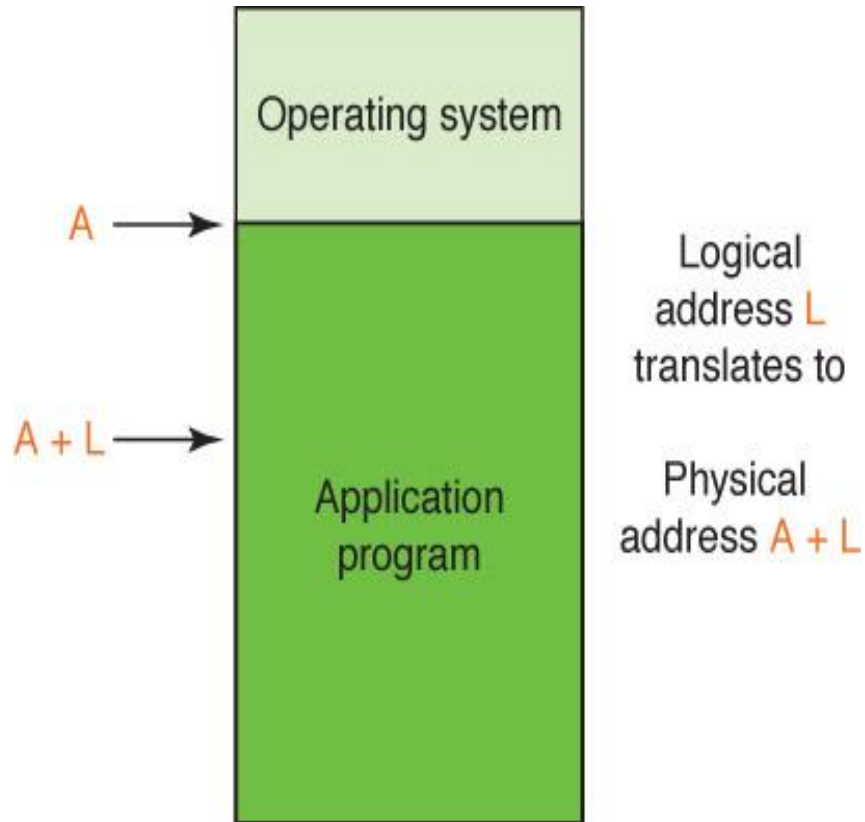
Single Contiguous MM

In concrete terms:

A **logical address** is simply an **integer** value relative to the **starting point** of the program

A **physical address** is a logical address added to the starting location of the program in main memory

Single Contiguous MM



If A is location 100, and the application program is Program 1, then sum is stored at location 123.

Figure Binding a logical address to a physical one

Partition Memory Management

Single contiguous MM has only the OS and one other program in memory at one time

Partition MM has the OS and any number of other programs in memory at one time

There are two schemes for dividing up memory for programs:

- **Fixed partitions** Main memory is divided into a fixed number of partitions into which programs can be loaded
- **Dynamic partitions** Partitions are created as needed to fit the programs waiting to be loaded

Partition Memory Management

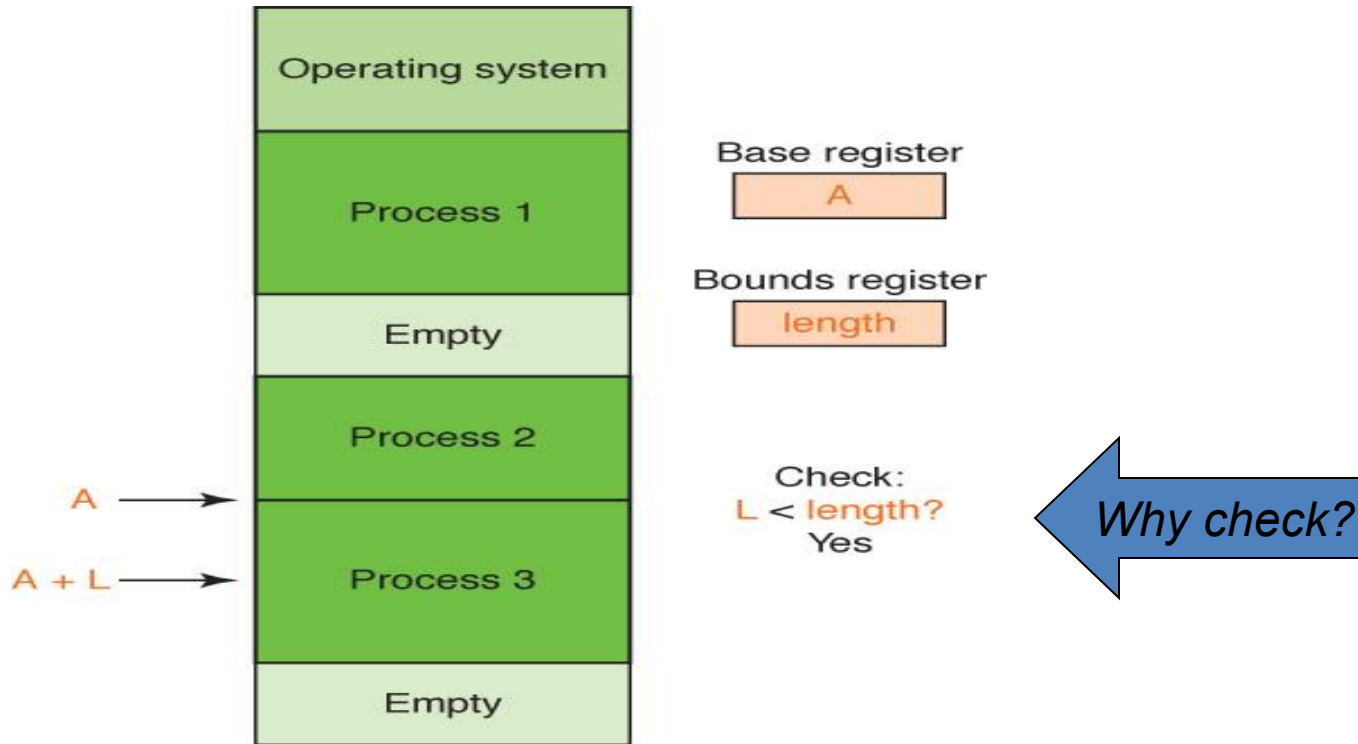


Figure
Address resolution
in partition memory
management

Partition Selection Algorithms

Which partition should we allocate to a new program?

- **First fit** Allocate program to the first partition big enough to hold it
- **Best fit** Allocated program to the smallest partition big enough to hold it
- **Worst fit** Allocate program to the largest partition big enough to hold it

Paged Memory Management

Paged memory technique

A technique in which processes are divided into fixed-size **pages** and stored in memory **frames** when loaded

Frame

A fixed-size portion of *main memory* that holds a process page

Page

A fixed-size portion of a *process* that is stored into a memory frame

We assume that a frame and a page are the same size

Paged Memory Management

P1 PMT	
Page	Frame
0	5
1	12
2	15
3	7
4	22

P2 PMT	
Page	Frame
0	10
1	18
2	1
3	11

Memory	
Frame	Contents
0	
1	P2/Page2
2	
3	
4	
5	P1/Page0
6	
7	P1/Page3
8	
9	
10	P2/Page0
11	P2/Page3
12	P1/Page1
13	
14	
15	P1/Page2

—



Prog. 2, Page 2



Prog. 1, Page 3

If Prog. 1 is running and needs logical address 2566, how is the actual address calculated?

Figure 10.7

A paged memory management approach

Paged Memory Management

Demand paging

An extension of paged memory management in which pages are brought into memory on demand

Page swap

The act of bringing in a page from secondary memory, which often causes another page to be written back to secondary memory

Process Management

There is only one CPU and therefore only one set of CPU registers, which contain the values for the currently executing process

Each time a process is moved to the running state:

- Register values for the currently running process are stored into its PCB
- Its PCB is moved to the list of the state into which it goes
- Register values of the new process moving into the running state are loaded into the CPU
- This exchange of register information is called a **context switch**

CPU Scheduling

CPU Scheduling

The act of determining which process in the *ready* state should be moved to the *running* state

- Many processes may be in the ready state
- Only one process can be in the running state, making progress at any one time

Which one gets to move from ready to running?

CPU Scheduling

Nonpreemptive scheduling

The currently executing process gives up the CPU voluntarily

Preemptive scheduling

The operating system decides to favor another process, preempting the currently executing process

Turnaround time

The amount of time between when a process arrives in the ready state the first time and when it exits the running state for the last time

CPU Scheduling Algorithms

First-Come, First-Served

Processes are moved to the CPU in the order in which they arrive in the running state

Shortest Job Next

Process with shortest estimated running time in the ready state is moved into the running state first

Round Robin

Each process runs for a specified time slice and moves from the running state to the ready state to await its next turn if not finished